

Министерство науки и высшего образования и науки Российской Федерации
Муромский институт (филиал)
федерального государственного бюджетного образовательного учреждения
высшего образования
**«Владимирский государственный университет
имени Александра Григорьевича и Николая Григорьевича Столетовых»**
(МИ ВлГУ)

Факультет _____ ИТР _____

Кафедра: _____ ИС _____

КУРСОВАЯ РАБОТА

по курсу _____ прикладная разработка на Java _____

на тему: _____ разработка веб-приложения для онлайн-голосования с
использованием Spring Boot _____

Руководитель

(Оценка)

_____ Метелкин А.С. _____

(фамилия, инициалы)

(подпись)

(дата)

Члены комиссии

Студент _____ ИС-123 _____

(группа)

(подпись, Ф.И.О.)

_____ Монахов И.А. _____

(фамилия, инициалы)

(подпись, Ф.И.О.)

(подпись)

(дата)

Содержание

Введение.....	6
1 Анализ технического задания.....	7
1.1 Функциональные требования	8
1.2 Нефункциональные требования	9
1.3 Выбор технологии разработки.....	9
2 Проектирование программы	13
2.1 Архитектура веб-приложения.....	13
2.2 Описание уровней архитектуры	13
2.3 Проектирование базы данных	15
3 Разработка приложения	17
3.1 Создание главной страницы и публичной части	17
3.2 Реализация административной панели	17
3.3 Реализация добавления опроса через форму	18
3.4 Реализация генерации временных ссылок	18
3.5 Реализация голосования по временной ссылке	19
3.6 Реализация отображения результатов.....	19
4 Тестирование	21
4.1 Тестирование компонента Builder для конструирования опросов	21
4.2 Тестирование компонента Factory для создания типовых опросов.....	23
4.3 Тестирование компонента временных ссылок	23
4.4 Тестирование компонента Observer	24
4.5 Тестирование сервисного слоя	25
Заключение	26
Список используемой литературы	28

					МИВУ 09.03.02-12.000 ПЗ			
Изм.	Лист	№ докум.	Подпись	Дата				
Разраб.		Монахов И.А.			Курсовой проект по теме: Разработка веб-приложения для онлайн-голосования	Лит.	Лист	Листов
Провер.		Метелкин А.С.					5	56
Реценз.						МИВлГУ ИС-123		
Н. Контр.								
Утверд.								

Введение

Современные информационные системы все чаще используют механизмы сбора и анализа общественного мнения для принятия обоснованных решений в различных сферах деятельности — от маркетинговых исследований до оценки качества предоставляемых услуг. Системы онлайн-голосования и опросов представляют собой комплексные программные решения, основу которых составляют базы данных, хранящие информацию о проводимых опросах, вопросах, вариантах ответов, голосах участников и временных ссылках доступа. Управление данными в таких системах является критически важным аспектом, определяющим достоверность результатов, производительность при высоких нагрузках и безопасность персональной информации респондентов.

Целью данной курсовой работы является разработка системы онлайн-голосования с акцентом на эффективное управление данными и реализацию современных архитектурных паттернов. Разработанная система должна обеспечивать хранение, обработку и защиту разнообразных данных, включая информацию об опросах различных типов, множественных вопросах с вариативными ответами, голосах пользователей с возможностью указания демографических характеристик, а также временных ссылках для контролируемого доступа.

В процессе разработки применялись принципы проектирования баз данных, включая нормализацию для устранения избыточности, создание индексов для ускорения поисковых операций и обеспечение ACID-свойств транзакций при сохранении голосов. Архитектура системы базируется на фреймворке Spring Boot с реализацией классических паттернов проектирования: Builder для конструирования сложных объектов опросов, Factory для создания типовых шаблонов, Observer для событийного логирования голосования и Strategy для инкапсуляции алгоритмов обработки голосов.

1 Анализ технического задания

Разработка системы онлайн-голосования представляет собой задачу создания интерактивного веб-приложения, реализующего функционал создания и проведения опросов, в котором администратор управляет опросами и генерирует временные ссылки для голосования, а пользователи участвуют в голосовании по этим ссылкам. Основные функциональные требования включают реализацию механики создания опросов с произвольным количеством вопросов и вариантов ответов, поддержку типовых шаблонов опросов (удовлетворённости, обратной связи, предпочтений), генерацию временных ссылок с настраиваемыми ограничениями по сроку действия и максимальному количеству голосов, обработку голосования с возможностью указания возраста респондента, а также обеспечение удобного пользовательского интерфейса с поддержкой трёх шаблонизаторов (Thymeleaf, JSP, FreeMarker) для демонстрации гибкости Spring MVC.

Система должна быть реализована на языке программирования Java с использованием фреймворка Spring Boot для backend-части и Firebird SQL для хранения данных. Архитектура приложения должна соответствовать принципам объектно-ориентированного программирования, обеспечивая модульность, расширяемость и повторное использование кода. Приложение должно поддерживать аутентификацию администратора с безопасным хешированием пароля, отображение списка доступных опросов на главной странице, а также просмотр статистики голосования с процентным соотношением ответов и возрастной информацией участников. Техническое задание подразумевает использование паттернов проектирования (Builder, Factory, Observer, Strategy) для конструирования опросов, создания типовых объектов, оповещения подписчиков о событиях голосования и инкапсуляции алгоритмов обработки голосов[3].

					<i>МИВУ.09.03.02-12.000 ПЗ</i>	Лист
Изм.	Лист	№ докум.	Подп.	Дата		7

Ключевые нефункциональные требования включают обеспечение стабильной работы приложения при одновременной нагрузке до 50 пользователей и защиту административной панели от несанкционированного доступа. Интерфейс должен быть интуитивно понятным, с чётким отображением списка опросов, вопросов с вариантами ответов и результатов голосования в виде прогресс-баров и процентных соотношений.

1.1 Функциональные требования

Система онлайн-голосования должна обеспечивать хранение иерархических данных: опросы содержат вопросы (polls), каждый вопрос содержит варианты ответов (options). Администратор должен иметь возможность аутентифицироваться в системе с использованием логина и пароля с безопасным хешированием, создавать новые опросы с произвольным названием, добавлять неограниченное количество вопросов и вариантов ответов. Система должна предоставлять возможность быстрого создания типовых опросов трёх видов: опрос удовлетворённости (с вопросами о качестве обслуживания, скорости работы и готовности рекомендовать), опрос обратной связи (о предпочтениях и направлениях улучшения) и опрос предпочтений (о формате и времени проведения). Генерация временных ссылок должна осуществляться с настраиваемыми параметрами: срок действия в часах (от 1 до 720) и максимальное количество голосов по ссылке (опционально). Каждая ссылка должна иметь уникальный токен длиной 8 символов, а система должна отслеживать количество использований ссылки и её статус (активна, истекла, заполнена).

Голосование должно осуществляться только по временным ссылкам. При переходе по ссылке система проверяет её валидность: срок действия не истёк, лимит голосов не превышен. Пользователь должен иметь возможность выбрать один вариант ответа на каждый вопрос опроса и опционально указать свой возраст. После сохранения голосов система увеличивает счётчик

использований ссылки и перенаправляет пользователя на страницу с результатами.

Отображение результатов должно включать подсчёт количества голосов за каждый вариант ответа, расчёт процентного соотношения относительно общего числа голосов по вопросу и визуализацию с помощью прогресс-баров. В административной панели дополнительно отображается возрастная статистика участников.

1.2 Нефункциональные требования

Приложение должно обеспечивать плавную работу интерфейса со временем отклика не более 2 секунд при стандартных операциях. Система должна быть надёжной, корректно обрабатывать ошибочные ситуации (несуществующий опрос, недействительная ссылка, истекший срок действия) и выдавать понятные пользователю сообщения. Интерфейс должен быть интуитивно понятным, с чёткими визуальными элементами.

1.3 Выбор технологии разработки

1.3.1 Spring Boot

Spring Boot представляет собой современный фреймворк для создания веб-приложений на Java, предоставляющий встроенный DI-контейнер, модуль Spring Security для аутентификации и авторизации, автоматическую конфигурацию и встроенный веб-сервер Tomcat. Его преимуществами являются высокая скорость разработки, обширная документация и интеграция с различными шаблонизаторами (Thymeleaf, JSP, FreeMarker), что делает его оптимальным выбором для веб-проектов, таких как система онлайн-голосования[2].

1.3.2 Thymeleaf

Thymeleaf — это современный шаблонизатор для Java, нативно интегрирующийся со Spring Boot и поддерживающий естественный HTML-синтаксис. Его преимущества: кэширование для рабочей среды, удобная работа со Spring-бинами и выражениями, а также полная поддержка HTML5. Однако Thymeleaf требует подключения дополнительной зависимости, в отличие от JSP, который встроен в Java EE, но эта зависимость легко управляется через Maven.

1.3.3 JSP

JSP — это традиционная технология шаблонизации Java, которая встроена в платформу Java EE и не требует дополнительных зависимостей. Её главное преимущество — простота и привычность для многих разработчиков. Однако JSP уступает Thymeleaf в плане интеграции со Spring Boot и не поддерживает такие современные функции, как фрагменты макетов и расширенную интерполяцию выражений.

1.3.4 FreeMaker

FreeMarker — это мощный шаблонизатор, не зависящий от сервлетов и обеспечивающий строгое разделение логики и представления. Он предоставляет высокую производительность и гибкость, но требует изучения собственного синтаксиса, отличного от HTML, что может увеличить время разработки.

1.3.5 Итог и выбор

Для системы онлайн-голосования в качестве основного шаблонизатора была выбрана технология Thymeleaf, так как она обеспечивает оптимальный баланс между простотой разработки и интеграцией со Spring Boot, предоставляя естественный синтаксис HTML и удобную работу с данными модели. JSP и FreeMarker добавлены как альтернативные движки для

демонстрации гибкости Spring MVC и иллюстрации разных подходов к рендерингу представлений.

В качестве фреймворка выбран Spring Boot 3.x, который предоставляет все необходимые компоненты для разработки веб-приложения с аутентификацией, работой с базами данных и поддержкой нескольких шаблонизаторов. Для хранения данных используется RDB SQL как лёгковесная встраиваемая СУБД с официальным JDBC-драйвером Jaybird, обеспечивающим полную совместимость с Java-приложениями.

1.4 Сравнение с аналогами

Для оценки разрабатываемой системы онлайн-голосования был проведён сравнительный анализ с существующими аналогами. В качестве объектов сравнения выбраны популярные сервисы для создания опросов: Google Forms, SurveyMonkey и Yandex.Forms.

Критерии сравнения:

1. Способ распространения опроса — возможность предоставления ограниченного доступа к голосованию.
2. Анонимность участников — необходимость регистрации для участия в опросе.
3. Настройка ограничений доступа — возможность задать срок действия ссылки и лимит голосов.
4. Демографическая статистика — сбор и отображение информации о возрасте респондентов.
5. Стоимость — бесплатность использования.
6. Открытость кода — возможность самостоятельного развёртывания и модификации.

Таблица 1. Результаты сравнительного анализа

Критерий	Google Forms	SurveyMonkey	Yandex.Forms	Моя система
Распространение по ссылке	Да	Да	Да	Да
Временные ссылки	Нет	Нет	Нет	Да
Ограничение по сроку действия	Нет	Частично	Нет	Да
Ограничение по количеству голосов	Нет	Нет	Нет	Да
Анонимность участников	Требуется аккаунт	Требуется аккаунт	Требуется аккаунт	Полностью анонимно
Возрастная статистика	Нет	Нет	Нет	Да
Бесплатное использование	Частично	Платная	Бесплатно	Полностью бесплатно
Открытый исходный код	Нет	Нет	Нет	Да

2 Проектирование программы

2.1 Архитектура веб-приложения

Перед началом разработки была определена общая структура веб-приложения. Для удобства сопровождения и расширения проект разделён на несколько уровней. Каждый уровень выполняет свою задачу и взаимодействует с другими уровнями через классы и интерфейсы.

Общая архитектура приложения имеет следующий вид:

Пользователь - Шаблонизаторы - Контроллеры – Сервисы – Паттерны – Репозитории – База данных

На рисунке 1 представлена общая архитектура веб-приложения.

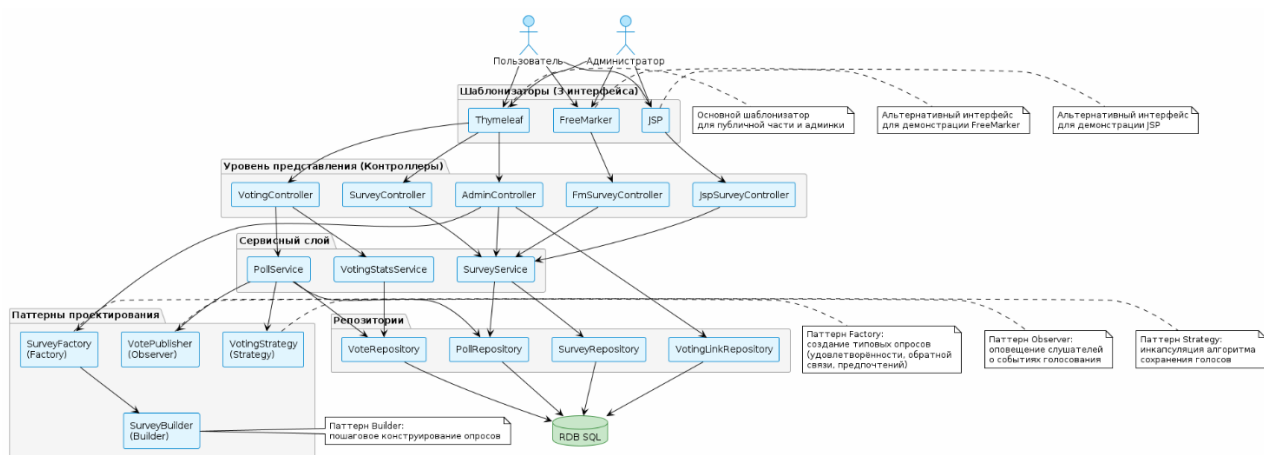


Рисунок 1 – Общая архитектура веб-приложения

2.2 Описание уровней архитектуры

Шаблонизаторы (3 интерфейса) — уровень представления, реализованный с использованием трёх шаблонизаторов. Thymeleaf используется в качестве основного интерфейса для публичной части и административной панели. FreeMarker и JSP являются альтернативными интерфейсами для демонстрации гибкости Spring MVC. Все три варианта

интерфейса используют общую программную логику и взаимодействуют с одними и теми же данными.

Уровень представления (Контроллеры) включает SurveyController для публичной части (главная страница и страница опроса), VotingController для голосования по временным ссылкам и отображения результатов, AdminController для административной панели (создание опросов, генерация ссылок), а также FmSurveyController и JspSurveyController для демонстрации работы с FreeMarker и JSP.

Сервисный слой содержит SurveyService для управления опросами, PollService для обработки голосования и VotingStatsService для подсчёта статистики. Сервисы реализуют основную бизнес-логику приложения и делегируют операции с данными репозиториям.

Паттерны проектирования включают четыре паттерна. SurveyFactory (Factory) создаёт типовые опросы трёх видов: удовлетворённости, обратной связи и предпочтений. VotePublisher (Observer) обеспечивает событийное оповещение слушателей о голосованиях. VotingStrategy (Strategy) инкапсулирует алгоритм сохранения голосов. SurveyBuilder (Builder) реализует пошаговое конструирование сложных объектов опросов с проверкой корректности данных.

Репозитории обеспечивают доступ к базе данных через JdbcTemplate. SurveyRepository управляет опросами, PollRepository — вопросами и вариантами ответов, VoteRepository — голосами, VotingLinkRepository — временными ссылками. Репозитории инкапсулируют SQL-запросы и преобразование результатов в объекты Java.

База данных RDB SQL хранит информацию об опросах (SURVEY), вопросах (POLL), вариантах ответов (OPTION), голосах (VOTE) и временных ссылках (VOTING_LINK). Все связи между таблицами организованы с использованием внешних ключей и каскадных операций для обеспечения целостности данных.

Изм.	Лист	№ докум.	Подп.	Дата

МИВУ.09.03.02-12.000 ПЗ

Лист

14

2.3 Проектирование базы данных

Для хранения данных используется реляционная база данных RDB SQL. В базе данных предусмотрены пять основных таблиц:

1. SURVEY (опросы);
2. POLL (вопросы);
3. OPTION (варианты ответов);
4. VOTE (голоса);
5. VOTING_LINK (временные ссылки).

Таблица SURVEY предназначена для хранения информации об опросах. В ней содержатся идентификатор опроса и название опроса. Поле TITLE является обязательным и не может быть пустым.

Таблица POLL предназначена для хранения информации о вопросах опроса. В ней содержатся идентификатор вопроса, текст вопроса и идентификатор опроса, к которому относится вопрос. Поле SURVEY_ID является внешним ключом, связывающим вопрос с опросом.

Таблица OPTION предназначена для хранения информации о вариантах ответов. Название таблицы заключено в кавычки, так как OPTION является зарезервированным словом в Firebird. В таблице содержатся идентификатор варианта ответа, текст варианта и идентификатор вопроса, к которому относится вариант. Поле POLL_ID является внешним ключом, связывающим вариант ответа с вопросом.

Таблица VOTE предназначена для хранения информации о голосах пользователей. В ней содержатся идентификатор голоса, возраст пользователя (может быть не указан) и идентификатор варианта ответа, за который был отдан голос. Поле OPTION_ID является внешним ключом, связывающим голос с вариантом ответа.

Таблица VOTING_LINK предназначена для хранения информации о временных ссылках для голосования. В ней содержатся идентификатор ссылки, уникальный токен, идентификатор опроса, дата и время истечения

срока действия, максимальное количество голосов (если ограничение не установлено, поле может быть пустым) и текущее количество использований ссылки. Поле SURVEY_ID является внешним ключом, связывающим ссылку с опросом[4].

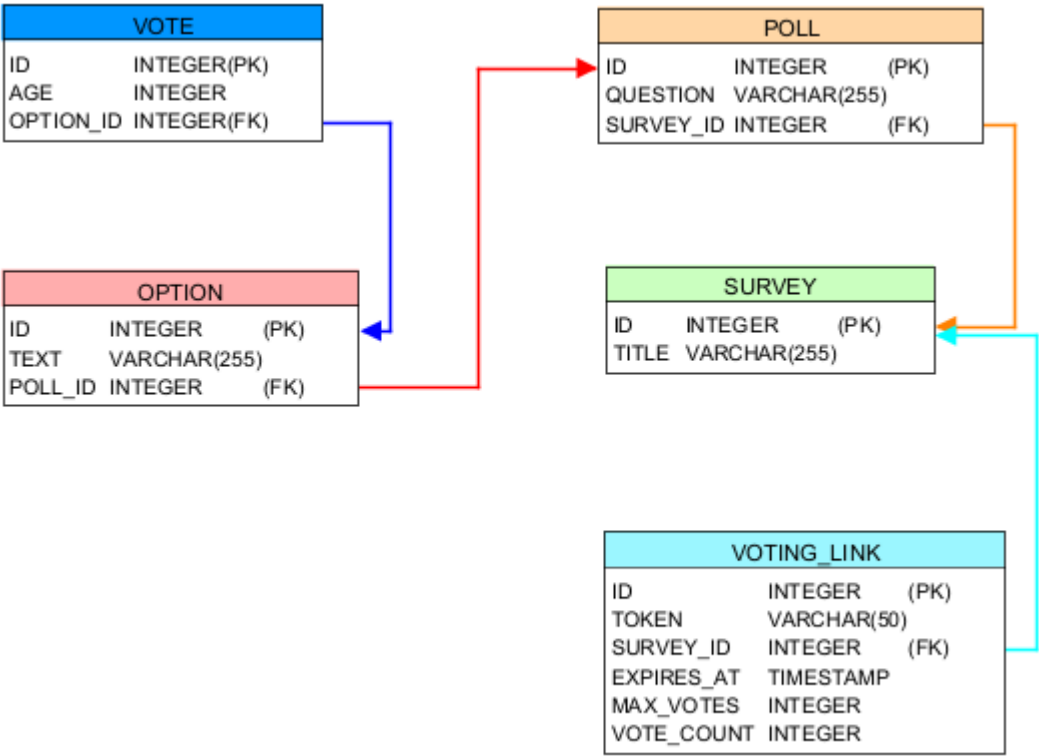


Рисунок 2 – ER-диаграмма базы данных

3 Разработка приложения

3.1 Создание главной страницы и публичной части

Разработка началась с создания компонента публичной части приложения, реализованного в классе `SurveyController`, который обрабатывает запросы к главной странице и страницам опросов. Для отображения используется шаблонизатор `Thymeleaf`, где список всех доступных опросов передаётся через модель и отображается в виде карточек. Каждая карточка содержит название опроса, количество вопросов и кнопку для перехода к голосованию. На главной странице также размещена кнопка входа в административную панель и виджет переключения между тремя шаблонизаторами. Эти элементы создали базовую структуру для дальнейшей реализации функционала.

3.2 Реализация административной панели

Создание компонентов для администратора начиналось с определения класса `AdminController` с базовым путём `/admin`. Все методы контроллера защищены аннотацией `@PreAuthorize("hasRole('ADMIN')")`, что обеспечивает доступ только после аутентификации. Метод `adminDashboard()` отображает дашборд, где для каждого опроса показываются все вопросы, варианты ответов и возрастная статистика проголосовавших пользователей. Метод `addSurveyForm()` отображает форму для создания нового опроса с динамическим добавлением вопросов и вариантов ответов через `JavaScript`. Метод `createSampleSurvey(@PathVariable String type)` позволяет создать типовой опрос в один клик: для типа `SATISFACTION` создаётся опрос удовлетворённости с вопросами о качестве обслуживания и скорости работы, для `FEEDBACK` — опрос обратной связи, для `PREFERENCE` — опрос предпочтений. Эти компоненты интегрируются с `SurveyFactory` и `SurveyBuilder` для конструирования опросов.

					<i>МИВУ.09.03.02-12.000 ПЗ</i>	Лист
Изм.	Лист	№ докум.	Подп.	Дата		17

3.3 Реализация добавления опроса через форму

Метод `addSurvey()` обрабатывает POST-запрос на создание опроса. Он принимает название опроса, список вопросов и список всех вариантов ответов. В цикле для каждого вопроса извлекаются до четырёх вариантов ответов (пользователь может добавить от 2 до 4 вариантов). Каждый вариант проверяется на пустоту, после чего через `SurveyBuilder` последовательно добавляются вопросы с вариантами. После завершения построения вызывается `surveyService.saveSurvey()` для сохранения в базу данных с каскадным сохранением всех вопросов и вариантов ответов. При успешном создании выполняется редирект на административную панель.

3.4 Реализация генерации временных ссылок

Для контролируемого доступа к голосованию был реализован механизм временных ссылок в классе `AdminController`. Метод `manageLinks()` отображает страницу управления ссылками для конкретного опроса, где показаны все ранее созданные ссылки с их статусом (активна, истекла, заполнена). Метод `createVotingLink()` генерирует уникальный токен длиной 8 символов через `UUID.randomUUID().toString().substring(0, 8)`. Пользователь может настроить срок действия ссылки в часах (от 1 до 720, по умолчанию 24) и максимальное количество голосов (опционально). Ссылка сохраняется в таблицу `VOTING_LINK` с указанием даты истечения и лимита голосов. После сохранения администратору показывается полная ссылка вида `http://localhost:8080/vote/abc12345`, которую можно скопировать и отправить респондентам.

3.5 Реализация голосования по временной ссылке

Класс `VotingController` реализует функционал голосования по временным ссылкам. Метод `voteByLink()` обрабатывает GET-запрос `/vote/{token}`, выполняет поиск ссылки по токену через `VotingLinkRepository`, проверяет её активность методом `isActive()` (срок не истёк и лимит голосов не превышен). При недействительности ссылки отображается страница `link-expired` с сообщением об ошибке. При валидной ссылке загружается соответствующий опрос через `SurveyService` и отображается страница голосования со всеми вопросами и вариантами ответов. Пользователь может выбрать один вариант на каждый вопрос и опционально указать свой возраст. Метод `submitVote()` обрабатывает POST-запрос, извлекает из параметров выбранные варианты ответов (параметры, начинающиеся с `poll_`), для каждого вызывает `pollService.vote()`. Внутри метода `vote()` сначала через `VotingStrategy` сохраняется голос в базу данных, затем через `VotePublisher` уведомляются подписчики (`LoggerListener` для логирования и `StatsListener` для сбора статистики). После сохранения всех голосов увеличивается счётчик использований ссылки через `votingLinkRepository.incrementVoteCount()` и выполняется редирект на страницу результатов[1].

3.6 Реализация отображения результатов

Метод `showResults()` отображает результаты голосования. Через `VotingStatsService` для каждого вопроса опроса получается список результатов: для каждого варианта ответа подсчитывается количество голосов через `voteRepository.countByOptionId()` и вычисляется процентное соотношение относительно общего числа голосов по вопросу. Результаты отображаются в виде таблицы с количеством голосов и процентами, а также с визуальными прогресс-барами. На странице также отображается статус

Изм.	Лист	№ докум.	Подп.	Дата

МИВУ.09.03.02-12.000 ПЗ

Лист

19

ссылки (активна или истекла) и общее количество проголосовавших по данной ссылке. В административной панели дополнительно показывается возрастная статистика — для каждого варианта ответа выводятся возраста всех проголосовавших пользователей.

4 Тестирование

4.1 Тестирование компонента Builder для конструирования опросов

Первым этапом было тестирование компонента SurveyBuilder, который отвечает за пошаговое создание опросов. Unit-тесты, написанные в классе SurveyBuilderTest, проверили установку названия опроса методом testSetTitle_ShouldSetTitle(), убедившись, что метод возвращает текущий экземпляр билдера для обеспечения цепочки вызовов. Далее в методе testAddPoll_WithValidOptions_ShouldAddPoll() тестировалось добавление вопроса с тремя вариантами ответов, проверялось, что вопрос корректно добавляется и количество вариантов соответствует ожидаемому.

```
@Test new *
void testAddPoll_WithValidOptions_ShouldAddPoll() {
    List<String> options = List.of("Вариант 1", "Вариант 2", "Вариант 3");

    surveyBuilder.setTitle("Мой опрос");
    surveyBuilder.addPoll(question: "Какой ваш любимый цвет?", options);
    Survey survey = surveyBuilder.build();

    assertEquals(expected: 1, survey.getPolls().size());
    assertEquals(expected: "Какой ваш любимый цвет?", survey.getPolls().get(0).getQuestion());
    assertEquals(expected: 3, survey.getPolls().get(0).getOptions().size());
}
```

Рисунок 3 – Тестирование добавления вопроса с вариантами ответов

Особое внимание уделялось фильтрации некорректных вариантов ответов. В тесте testAddPoll_WithNullOptions_ShouldSkipNulls() моделировалась ситуация, когда в списке вариантов присутствуют null и пустые строки. Проверялось, что такие значения пропускаются, и в итоговый опрос добавляются только валидные варианты.

```

@Test new *
void testAddPoll_WithNullOptions_ShouldSkipNulls() {
    List<String> options = List.of("Вариант 1", null, "Вариант 3", "");

    surveyBuilder.setTitle("Тест");
    surveyBuilder.addPoll( question: "Вопрос?", options);
    Survey survey = surveyBuilder.build();

    assertEquals( expected: 2, survey.getPolls().get(0).getOptions().size());
    assertEquals( expected: "Вариант 1", survey.getPolls().get(0).getOptions().get(0).getText());
    assertEquals( expected: "Вариант 3", survey.getPolls().get(0).getOptions().get(1).getText());
}

```

Рисунок 4 – Тестирование фильтрации некорректных вариантов

Также тестировалась валидация при построении опроса. Метод `testBuild_WithoutTitle_ShouldThrowException()` проверил, что при попытке построить опрос без названия выбрасывается исключение `IllegalStateException` с сообщением «Название опроса не может быть пустым». Аналогично метод `testBuild_WithoutPolls_ShouldThrowException()` проверил, что опрос без вопросов также не может быть построен.

```

@Test new *
void testBuild_WithoutTitle_ShouldThrowException() {
    surveyBuilder.addPoll( question: "Вопрос?", List.of("Опция 1", "Опция 2"));

    IllegalStateException exception = assertThrows(IllegalStateException.class, () -> {
        surveyBuilder.build();
    });
    assertEquals( expected: "Название опроса не может быть пустым", exception.getMessage());
}

@Test new *
void testBuild_WithEmptyTitle_ShouldThrowException() {
    surveyBuilder.setTitle("");
    surveyBuilder.addPoll( question: "Вопрос?", List.of("Опция 1", "Опция 2"));

    IllegalStateException exception = assertThrows(IllegalStateException.class, () -> {
        surveyBuilder.build();
    });
    assertEquals( expected: "Название опроса не может быть пустым", exception.getMessage());
}

```

Рисунок 5 - Тестирование валидации при построении опроса

4.2 Тестирование компонента Factory для создания типовых опросов

Следующим этапом стало тестирование компонента SurveyFactory, который создаёт предустановленные шаблоны опросов. Unit-тесты в классе SurveyFactoryTest с использованием Mockito проверили создание опроса удовлетворённости. Метод

testCreateSurvey_Satisfaction_ShouldCreateCorrectSurvey() подтвердил, что фабрика сбрасывает билдер, устанавливает правильное название и добавляет три вопроса с вариантами ответов.

```
@Test new *
void testCreateSurvey_Satisfaction_ShouldCreateCorrectSurvey() {
    Survey survey = surveyFactory.createSurvey(SurveyType.SATISFACTION, customTitle: null);

    assertNotNull(survey);
    verify(surveyBuilder, times(wantedNumberOfInvocations: 1)).reset();
    verify(surveyBuilder, times(wantedNumberOfInvocations: 1)).setTitle("Опрос удовлетворённости");
    verify(surveyBuilder, times(wantedNumberOfInvocations: 3)).addPollWithOptions(anyString(), any());
    verify(surveyBuilder, times(wantedNumberOfInvocations: 1)).build();
}
```

Рисунок 6 - Тестирование создания опроса удовлетворённости

4.3 Тестирование компонента временных ссылок

Далее проводилось тестирование класса VotingLink, который управляет временными ссылками для голосования. Метод testIsExpired_WhenExpired_ShouldReturnTrue() проверил определение истечения срока действия, а метод testIsFull_WhenMaxVotesReached_ShouldReturnTrue() – определение заполненности ссылки при достижении лимита голосов.

```

@Test new *
void testIsExpired_WhenExpired_ShouldReturnTrue() {
    VotingLink link = new VotingLink( token: "token", surveyId: 1L, LocalDateTime.now().minusHours(1), maxVotes: null);
    assertTrue(link.isExpired());
}

@Test new *
void testIsFull_WhenNoMaxVotes_ShouldReturnFalse() {
    VotingLink link = new VotingLink( token: "token", surveyId: 1L, LocalDateTime.now().plusHours(24), maxVotes: null);
    assertFalse(link.isFull());
}

@Test new *
void testIsFull_WhenMaxVotesReached_ShouldReturnTrue() {
    VotingLink link = new VotingLink( token: "token", surveyId: 1L, LocalDateTime.now().plusHours(24), maxVotes: 10);
    link.setVoteCount(10);
    assertTrue(link.isFull());
}

```

Рисунок 7 - Тестирование проверки истечения срока ссылки

4.4 Тестирование компонента Observer

Для проверки работы паттерна Observer были написаны тесты в классе VotePublisherTest.

Метод testSubscribe_AndNotifyListeners_ShouldCallAllListeners() проверил, что при вызове notifyListeners() все подписанные слушатели получают уведомление. Метод testUnsubscribe_ShouldRemoveListener() подтвердил, что после отписки слушатель перестаёт получать уведомления.

```

@Test new *
void testSubscribe_AndNotifyListeners_ShouldCallAllListeners() {
    votePublisher.subscribe(mockListener1);
    votePublisher.subscribe(mockListener2);

    votePublisher.notifyListeners( optionId: 100L, age: 25, pollId: 10L);

    verify(mockListener1, times( wantedNumberOfInvocations: 1)).onVoteCast( optionId: 100L, age: 25, pollId: 10L);
    verify(mockListener2, times( wantedNumberOfInvocations: 1)).onVoteCast( optionId: 100L, age: 25, pollId: 10L);
}

@Test new *
void testUnsubscribe_ShouldRemoveListener() {
    votePublisher.subscribe(mockListener1);
    votePublisher.subscribe(mockListener2);

    votePublisher.unsubscribe(mockListener1);
    votePublisher.notifyListeners( optionId: 100L, age: 25, pollId: 10L);

    verify(mockListener1, never()).onVoteCast(anyLong(), any(), anyLong());
    verify(mockListener2, times( wantedNumberOfInvocations: 1)).onVoteCast( optionId: 100L, age: 25, pollId: 10L);
}

```

Рисунок 8 - Тестирование публикатора событий голосования

4.5 Тестирование сервисного слоя

Для проверки работы сервисного слоя были написаны тесты в классе PollServiceImplTest.

Метод testVote_WithValidOption_ShouldCallStrategyAndPublisher() проверил, что при корректном варианте ответа сначала вызывается стратегия для сохранения голоса, а затем публикатор для оповещения слушателей. Метод testVote_WithOptionNotFound_ShouldCallStrategyButNotPublisher() подтвердил, что если вариант ответа не найден, публикатор не уведомляется.

```
@Test new *
void testVote_WithValidOption_ShouldCallStrategyAndPublisher() {
    when(pollRepository.findOptionById( optionId: 100L)).thenReturn(mockOption);

    pollService.vote( optionId: 100L, age: 25);

    verify(votingStrategy, times( wantedNumberOfInvocations: 1)).castVote( optionId: 100L, age: 25);
    verify(votePublisher, times( wantedNumberOfInvocations: 1)).notifyListeners( optionId: 100L, age: 25, pollId: 10L);
}

@Test new *
void testVote_WithOptionNotFound_ShouldCallStrategyButNotPublisher() {
    when(pollRepository.findOptionById( optionId: 200L)).thenReturn( t: null);

    pollService.vote( optionId: 200L, age: 30);

    verify(votingStrategy, times( wantedNumberOfInvocations: 1)).castVote( optionId: 200L, age: 30);
    verify(votePublisher, never()).notifyListeners(anyLong(), any(), anyLong());
}
```

Рисунок 9 - Тестирование сервиса голосования

Все тесты успешно выполнены, что подтверждает корректную работу основных компонентов системы онлайн-голосования: Builder для конструирования опросов, Factory для создания типовых шаблонов, Observer для событийного оповещения, Strategy для обработки голосования, а также моделей данных и сервисного слоя.

Заключение

В результате выполнения курсовой работы была создана система онлайн-голосования, полностью соответствующая требованиям технического задания. Разработанное приложение реализует полный цикл проведения опросов: от создания опросов администратором до участия пользователей и просмотра статистики. Система поддерживает создание опросов с произвольным количеством вопросов и вариантов ответов, генерацию временных ссылок с настраиваемыми ограничениями по сроку действия и максимальному количеству голосов, а также сбор демографической информации (возраст респондентов) для последующего статистического анализа.

В ходе разработки были реализованы четыре ключевых паттерна проектирования. Паттерн Builder позволяет пошагово конструировать сложные объекты опросов с проверкой корректности данных. Паттерн Factory обеспечивает быстрое создание типовых опросов трёх видов: удовлетворённости, обратной связи и предпочтений. Паттерн Observer реализует событийное оповещение подписчиков о голосованиях для логирования и сбора статистики. Паттерн Strategy инкапсулирует алгоритм сохранения голосов, что позволяет легко расширять функциональность в будущем.

Для хранения данных использована СУБД RDB. Разработанная схема базы данных включает пять основных сущностей: SURVEY (опросы), POLL (вопросы), OPTION (варианты ответов), VOTE (голоса) и VOTING_LINK (временные ссылки). Все связи организованы с использованием внешних ключей и каскадных операций, что обеспечивает целостность данных.

В процессе разработки были углублены и закреплены практические навыки работы с Java 17 и Spring Boot 3.x, включая создание веб-приложений с использованием MVC-архитектуры, внедрение зависимостей, настройку безопасности с Spring Security, работу с базами данных через JdbcTemplate.

					<i>МИВУ.09.03.02-12.000 ПЗ</i>	Лист
Изм.	Лист	№ докум.	Подп.	Дата		26

Освоены современные шаблонизаторы Thymeleaf, JSP и FreeMarker, что позволило продемонстрировать гибкость Spring MVC в вопросах рендеринга представлений. Применение JUnit и Mockito для unit-тестирования укрепило навыки обеспечения качества программного обеспечения. Работа в среде IntelliJ IDEA способствовала эффективной отладке кода, а система сборки Maven упростила управление зависимостями проекта.

Все поставленные цели достигнуты. Разработанная система может использоваться для проведения реальных опросов и голосований в учебных, маркетинговых или корпоративных целях, а также служить основой для дальнейшего развития с добавлением новых функций, таких как авторизация пользователей, экспорт статистики в различные форматы и визуализация результатов в виде диаграмм.

					<i>МИВУ.09.03.02-12.000 ПЗ</i>	Лист
						27
Изм.	Лист	№ докум.	Подп.	Дата		

Список используемой литературы

1. Шилдт, Г. Java 8. Полное руководство / Г. Шилдт. – 9-е изд. – М.: Вильямс, 2015. – 1376 с.
2. Хорстманн, К. С. Java. Библиотека профессионала. Том 2. Средства продвинутой разработки / К. С. Хорстманн. – 10-е изд. – М. : Вильямс, 2018. – 992 с.
3. Блинов, И. Н. Java. Методы программирования / И. Н. Блинов, В. С. Романчик. – Минск: Четыре четверти, 2013. – 896 с.
4. Документация по базе данных RedDataBase [Электронный ресурс]: справочное руководство / RedDataBase. — Электрон. текстовые данные. — Режим доступа: <https://www.red-database.com/documentation>

Изм.	Лист	№ докум.	Подп.	Дата

МИВУ.09.03.02-12.000 ПЗ

Лист

28